

자료구조 및 알고리즘

1. 정렬이란?

■ 정렬

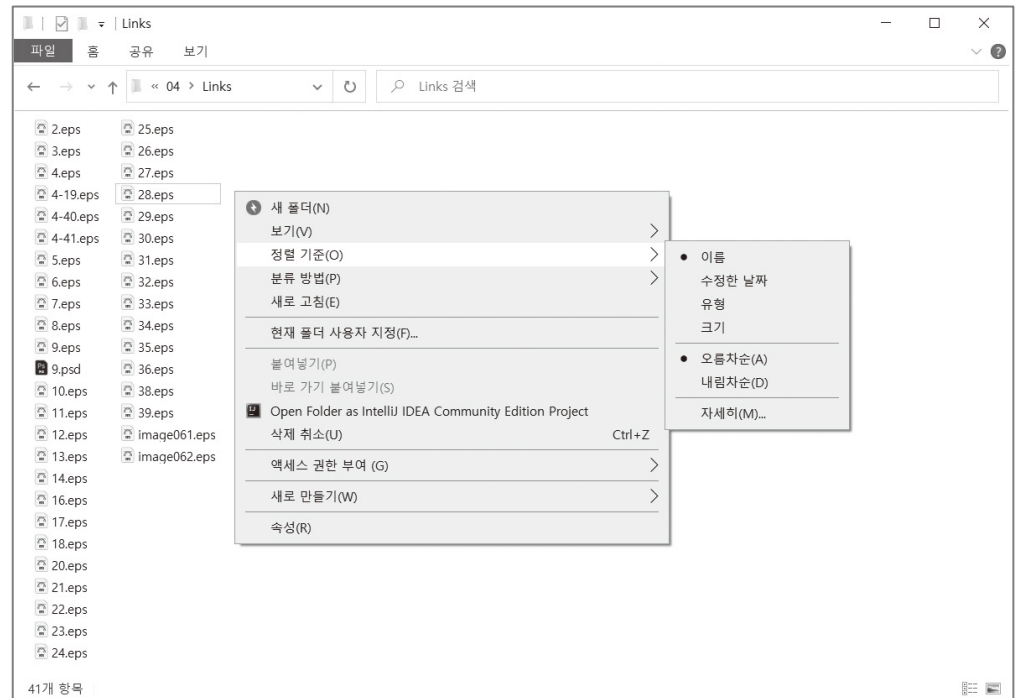
■ 원소들을 순서대로 배열하는 것

- 알파벳 순
- 학번 순
- 생년월일 순
- etc...

■ 기본 정렬

■ 고급 정렬

■ 특수 정렬



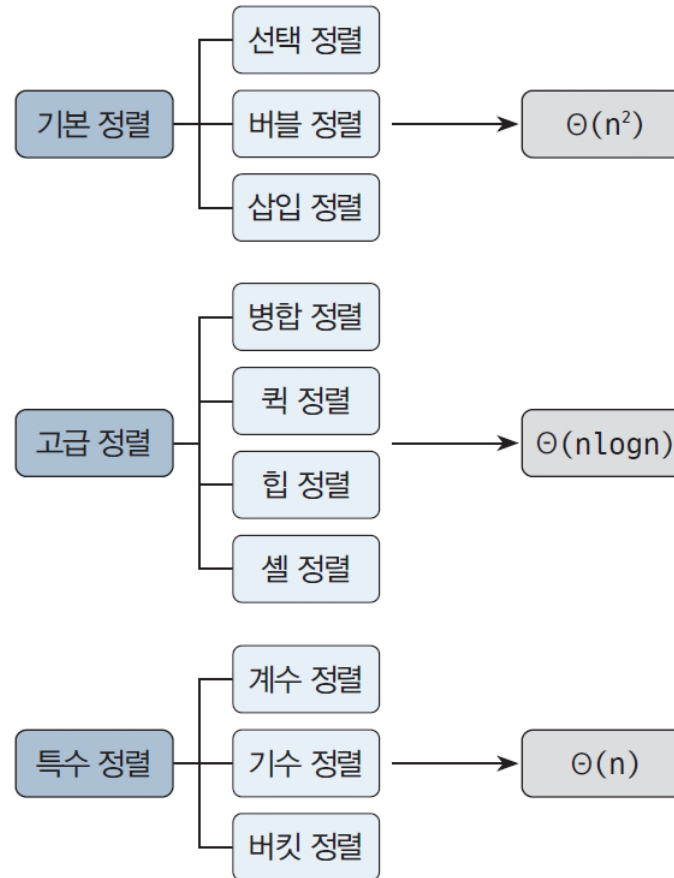


그림 9-2 이 장에서 소개하는 정렬 알고리즘과 평균 성능

- 정렬 알고리즘의 복잡도

- 대부분의 정렬 알고리즘은 $\Omega(n \log n)$ 과 $O(n^2)$ 사이
- 입력이 특수한 성질을 만족하는 경우 $\Theta(n)$ 도 가능

2. 기본 정렬 알고리즘

- 선택 정렬

```
selectionSort(A[], n):  
    for last ← n-1 down to 1  
        A[0 ... last] 중 가장 큰 수 A[k] 찾기  
        A[k] ↔ A[last]
```

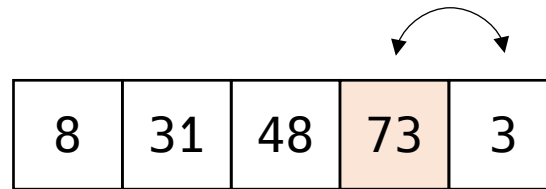
- 선택 정렬

8	31	48	73	3
---	----	----	----	---

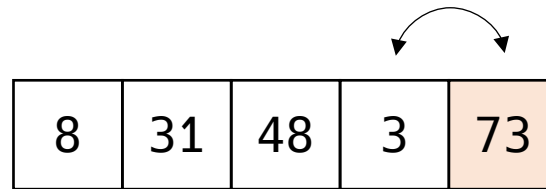
- 선택 정렬

8	31	48	73	3
---	----	----	----	---

- 선택 정렬



- 선택 정렬



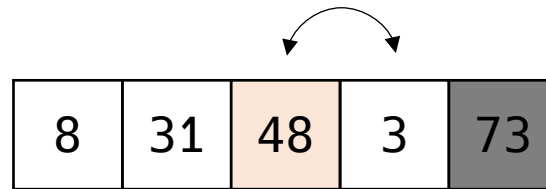
- 선택 정렬

8	31	48	3	73
---	----	----	---	----

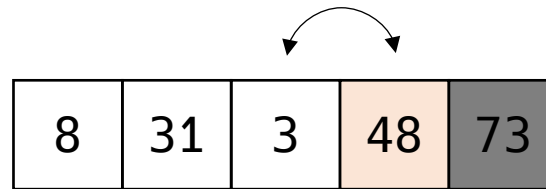
- 선택 정렬

8	31	48	3	73
---	----	----	---	----

- 선택 정렬



- 선택 정렬



- 선택 정렬

8	31	3	48	73
---	----	---	----	----

- 선택 정렬

3	8	31	48	73
---	---	----	----	----

- 선택 정렬

```
selectionSort(A[], n):  
    for last ← n-1 down to 1  
        A[0 ... last] 중 가장 큰 수 A[k] 찾기  
        A[k] ↔ A[last]
```

- 수행시간 (비교 횟수)
 - for 루프 : (n-1)번
 - 한 루프 당 비교 : (last-1) 번

→ $n(n-1)/2 = \Theta(n^2)$

■ 선택 정렬 구현

```
def selectionSort(A):
    for last in range(len(A)-1, 0, -1):
        k = theLargest(A, last)      # A[0...last] 중 가장 큰 수 A[k] 찾기
        A[k], A[last] = A[last], A[k]

def theLargest(A, last:int) -> int: # A[0...last]에서 가장 큰 수의 인덱스를 리턴한다
    largest = 0
    for i in range(last+1):
        if A[i] > A[largest]:
            largest = i
    return largest
```

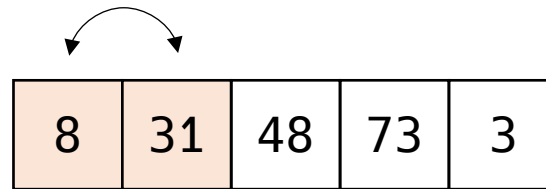
- 버블 정렬

```
bubbleSort(A[], n):  
    for last ← n-1 down to 1  
        for i ← 0 to last-1  
            if (A[i] > A[i+1])  
                A[i] ↔ A[i+1]
```

- 버블 정렬

8	31	48	73	3
---	----	----	----	---

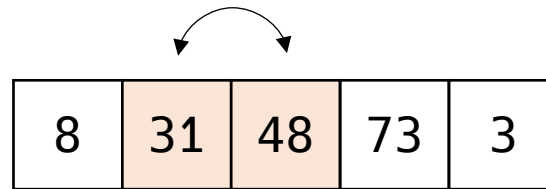
- 버블 정렬



- 버블 정렬

8	31	48	73	3
---	----	----	----	---

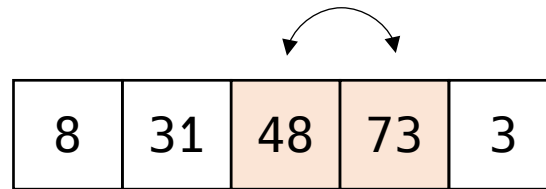
- 버블 정렬



- 버블 정렬

8	31	48	73	3
---	----	----	----	---

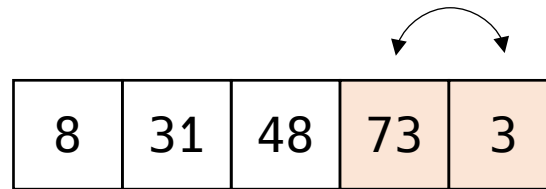
- 버블 정렬



- 버블 정렬

8	31	48	73	3
---	----	----	----	---

- 버블 정렬



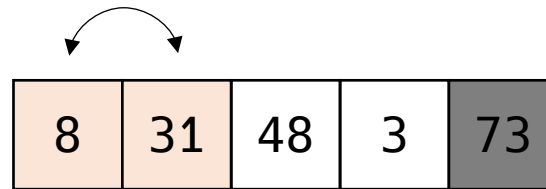
- 버블 정렬

8	31	48	3	73
---	----	----	---	----

- 버블 정렬

8	31	48	3	73
---	----	----	---	----

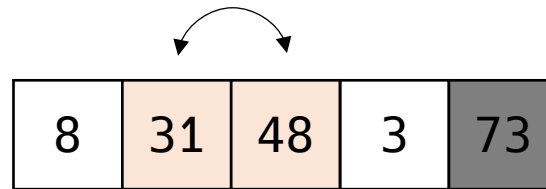
- 버블 정렬



- 버블 정렬

8	31	48	3	73
---	----	----	---	----

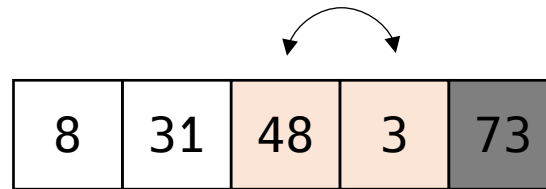
- 버블 정렬



- 버블 정렬

8	31	48	3	73
---	----	----	---	----

- 버블 정렬



- 버블 정렬

8	31	3	48	73
---	----	---	----	----

- 버블 정렬

8	31	3	48	73
---	----	---	----	----

- 버블 정렬

3	8	31	48	73
---	---	----	----	----

- 버블 정렬

```
bubbleSort(A[], n):  
    for last ← n-1 down to 1  
        for i ← 0 to last-1  
            if (A[i] > A[i+1])  
                A[i] ↔ A[i+1]
```

- 수행시간 (비교 횟수)
 - for 루프 : (n-1)번
 - 한 루프 당 비교 : (last-1) 번

→ $n(n-1)/2 = \Theta(n^2)$

- 버블 정렬 구현

```
def bubbleSort(A):  
    for numElements in range(len(A), 0, -1):  
        for i in range(numElements-1):  
            if A[i] > A[i+1]:  
                A[i], A[i+1] = A[i+1], A[i]
```

- 삽입 정렬

```
insertionSort(A[], n):  
    for i ← 1 to n-1  
        A[0 ... i]의 적합한 자리에 A[i]를 삽입
```

- 삽입 정렬

31	48	8	73	3
----	----	---	----	---

- 삽입 정렬

31	48	8	73	3
----	----	---	----	---

- 삽입 정렬

31	48	8	73	3
----	----	---	----	---

- 삽입 정렬

31	48	8	73	3
----	----	---	----	---

- 삽입 정렬

31	48	8	73	3
----	----	---	----	---

- 삽입 정렬

31	48	8	73	3
----	----	---	----	---

- 삽입 정렬

8	31	48	73	3
---	----	----	----	---

- 삽입 정렬

8	31	48	73	3
---	----	----	----	---

- 삽입 정렬

8	31	48	73	3
---	----	----	----	---

- 삽입 정렬

8	31	48	73	3
---	----	----	----	---

- 삽입 정렬

8	31	48	73	3
---	----	----	----	---

- 삽입 정렬

8	31	48	73	3
---	----	----	----	---

- 삽입 정렬

3	8	31	48	73
---	---	----	----	----

- 삽입 정렬

3	8	31	48	73
---	---	----	----	----

- 삽입 정렬

```
insertionSort(A[], n):  
  for i ← 1 to n-1  
    A[0 ... i]의 적합한 자리에 A[i]를 삽입
```

- 삽입 정렬

```
insertionSort2(A[], n):  
    for i ← 1 to n-1  
        j ← i-1  
        newItem = A[i]  
  
        while (0 ≤ j and newItem < A[j])  
            A[j+1] ← A[j]  
            j--  
        A[j+1] ← newItem
```

- 수행시간 (비교 횟수)

- for 루프 : (n-1)번
- 한 루프 당 비교 : 최대 (i-1)

→ 최악 : $n(n-1)/2 = \Theta(n^2)$ 최선 : $n-1 = \Theta(n) \rightarrow O(n^2)$

- 삽입 정렬 구현

```
def insertionSort(A):  
    for i in range(1, len(A)):  
        loc = i-1  
        newItem = A[i]  
        while loc >= 0 and newItem < A[loc]:  
            A[loc+1] = A[loc]  
            loc -= 1  
        A[loc+1] = newItem
```